실습 관련(Next Class)

- 언리얼 엔진 버전 – 5.4.4 이상
  - 웬만하면 5.4.4를 권장합니다. (버전이 높다고 무조건 좋은건 아님..)

- IDE : 원하시는 걸 쓰세요!
  - 본인은 Visual studio를 씀.
  - 하지만 Rider도 충분히 쓸 수 있을만큼 좋음! (맥북이라면 사실 필수)
  - Visual Code..? 할 수 있다면..

- 원격 조종 툴 : Parsec (만약 노트북 사양이 좋지 않다면)
  - 집에 있는 데스크탑에 설치.

언리얼 스터디 2주차

1. 우리는 지금 어디에 있죠?

- Unreal Engine 5.7 Documentation
  - What's New
  - Get Started
  - Understanding the Basics
  - Working with Content
  - Building Virtual Worlds
  - Designing Visuals, Rendering, and Graphics
  - Creating Visual Effects
  - Gameplay Tutorials
  - Blueprints Visual Scripting
  - Programming with C++
  - Gameplay Systems
    - Artificial Intelligence
    - Data Driven Gameplay Elements
    - Gameplay Ability System
    - Gameplay Architecture
    - Gameplay Camera System
    - Gameplay Targeting System
    - Input
    - Class Creation Basics
    - Gameplay Framework
      - Actors
      - Camera

- Game Features and Modular Gameplay
- Game Mode and Game State
- Pawn
- Gameplay Framework Quick Reference
- Gameplay Timers
- User Interfaces and HUDs
- Physics
- Large World Coordinates
- Online Subsystems and Services
- Networking and Multiplayer
- Mover
- Mobile Development
- Animating Characters and Objects
- Motion Design
- Creating User Interfaces
- Working with Audio
- Working with Media
- Setting Up Your Production Pipeline
- Testing and Optimizing Your Content
- Sharing and Releasing Projects
- Samples and Tutorials

- Unreal Engine Web API Documentation
- Unreal Engine Blueprint API Reference
- Unreal Engine C++ API Reference
- Unreal Engine Python API Documentation
- Unreal Engine Node References

1. 우리는 지금 어디에 있죠?

- Unreal Engine 5.7 Documentation
  - What's New
  - Get Started
  - Understanding the Basics
  - Working with Content
  - Building Virtual Worlds
  - Designing Visuals, Rendering, and Graphics
  - Creating Visual Effects
  - Gameplay Tutorials
  - Blueprints Visual Scripting
  - Programming with C++
  - Gameplay Systems
    - Artificial Intelligence
    - Data Driven Gameplay Elements
    - Gameplay Ability System
    - Gameplay Architecture
    - Gameplay Camera System
    - Gameplay Targeting System
    - Input
    - Class Creation Basics
    - Gameplay Framework
      - Actors
      - Camera

- Game Features and Modular Gameplay
- Game Mode and Game State
- Pawn
- Gameplay Framework Quick Reference
- Gameplay Timers
- User Interfaces and HUDs
- Physics
- Large World Coordinates
- Online Subsystems and Services
- Networking and Multiplayer
- Mover
- Mobile Development
- Animating Characters and Objects
- Motion Design
- Creating User Interfaces
- Working with Audio
- Working with Media
- Setting Up Your Production Pipeline
- Testing and Optimizing Your Content
- Sharing and Releasing Projects
- Samples and Tutorials

- Unreal Engine Web API Documentation
- Unreal Engine Blueprint API Reference
- Unreal Engine C++ API Reference
- Unreal Engine Python API Documentation
- Unreal Engine Node References

Related by..

- https://dev.epicgames.com/documentation/en-us/unreal-engine/game-mode-and-game-state-in-unreal-engine

- Gamemode / GameState!

Game Elements... (Things we used to make all at once)

Sound

Cheat Guard

Game Money

Story

Skill

Network

Character

Input Detect

Quest

Boss

Visual(Graphics) Optimizer

Player Stat

Raid

Save & Load

And Others....

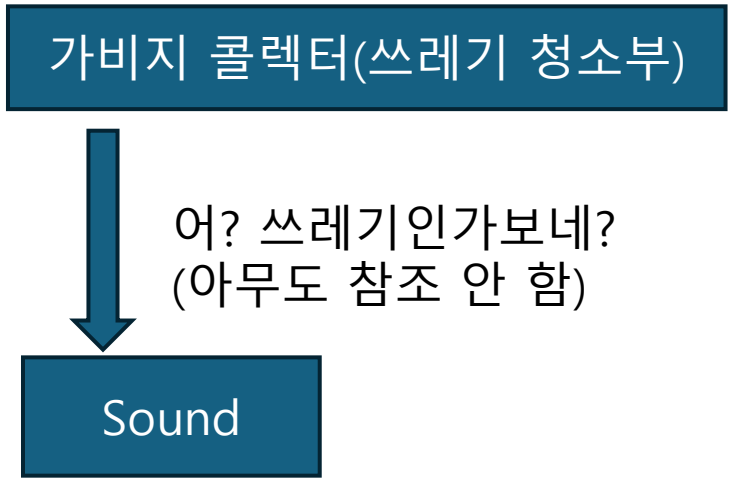
- 우리는 이보다 더 많은 것들을 다루고 싶으며, 또한 다룬다!

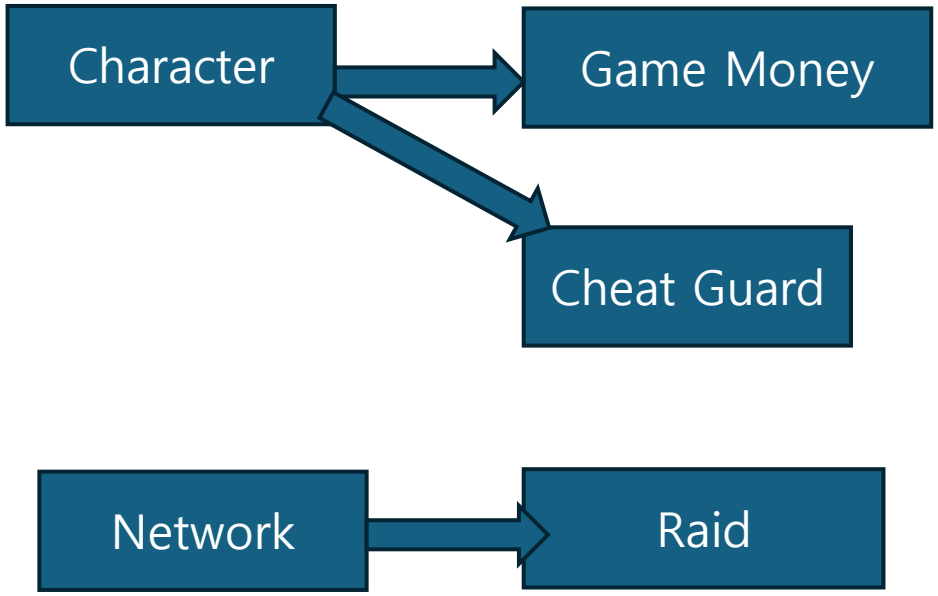
Problem 1. 누가 이걸 통제하죠?

- 게임 속 요소들은 서로 상호작용한다.
  - Input Detect -> Chracter에게 알려주어야 한다.
  - Cheat Guard -> Game Money를 계속 확인할 수 있어야 한다.
  - Network -> Player가 Server인지 Client인지 확인할 수 있어야 한다.

- 또한 생기는 타이밍, 사라지는 타이밍이 다르다.
  - 그리고 누가 누구의 참조를 가지고 있을지도 알 수 없다. (C++의 약점)
  - 만약 게임에 필수적인 기능이고, 다른 기능들과 상호작용 해야 하는데
  - 아무도 그 기능이 구현된 클래스를 가리키고 있지 않다면?

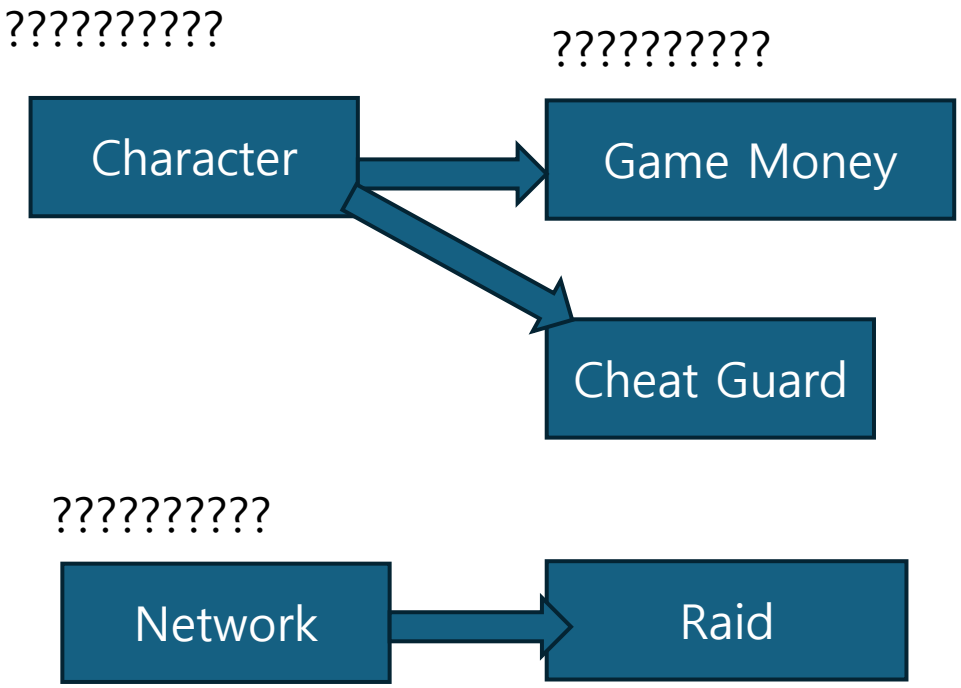
C++의 숙명


가비지 콜렉터(쓰레기 청소부)
어? 쓰레기인가보네?
(아무도 참조 안 함)
Sound


Character
Game Money
Cheat Guard
Network
Raid

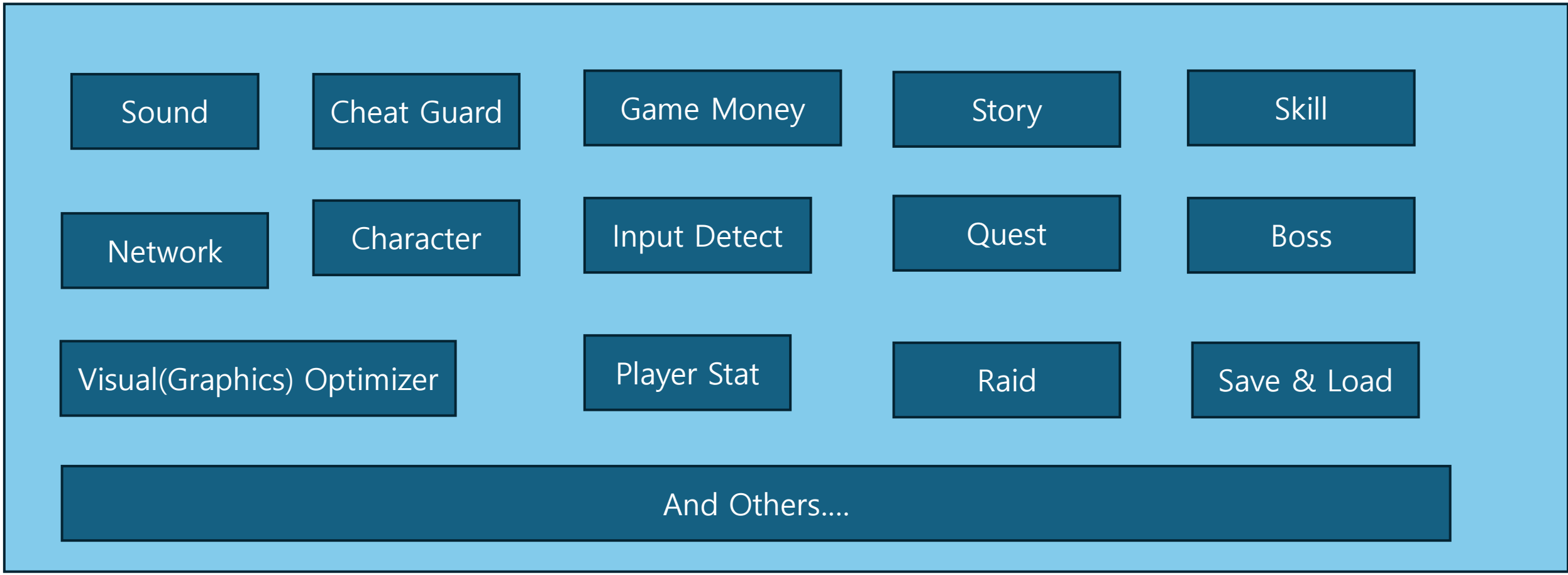
C++의 숙명

가져가야겠다~



- 참조하는 (가리키는) 클래스가 없으면 수거된다

GameManager (초기의)



- 모든 기능들을 참조하고 있는 클래스의 완성!

Problem 2. GameManager = God

Sound | Cheat Guard | Game Money | Story | Skill | r Stat

Network | Character | Input Detect | Quest | Raid | (Graphics) timizer

- 모든 변수는 다 GameManager에 넣기!(안전하니까)
- GamaManager가 다 초기화하게 만들기
- GameManager가 죄다 통신 도맡게 하기
- 새로 만드는 기능도 다 여기에 넣기!

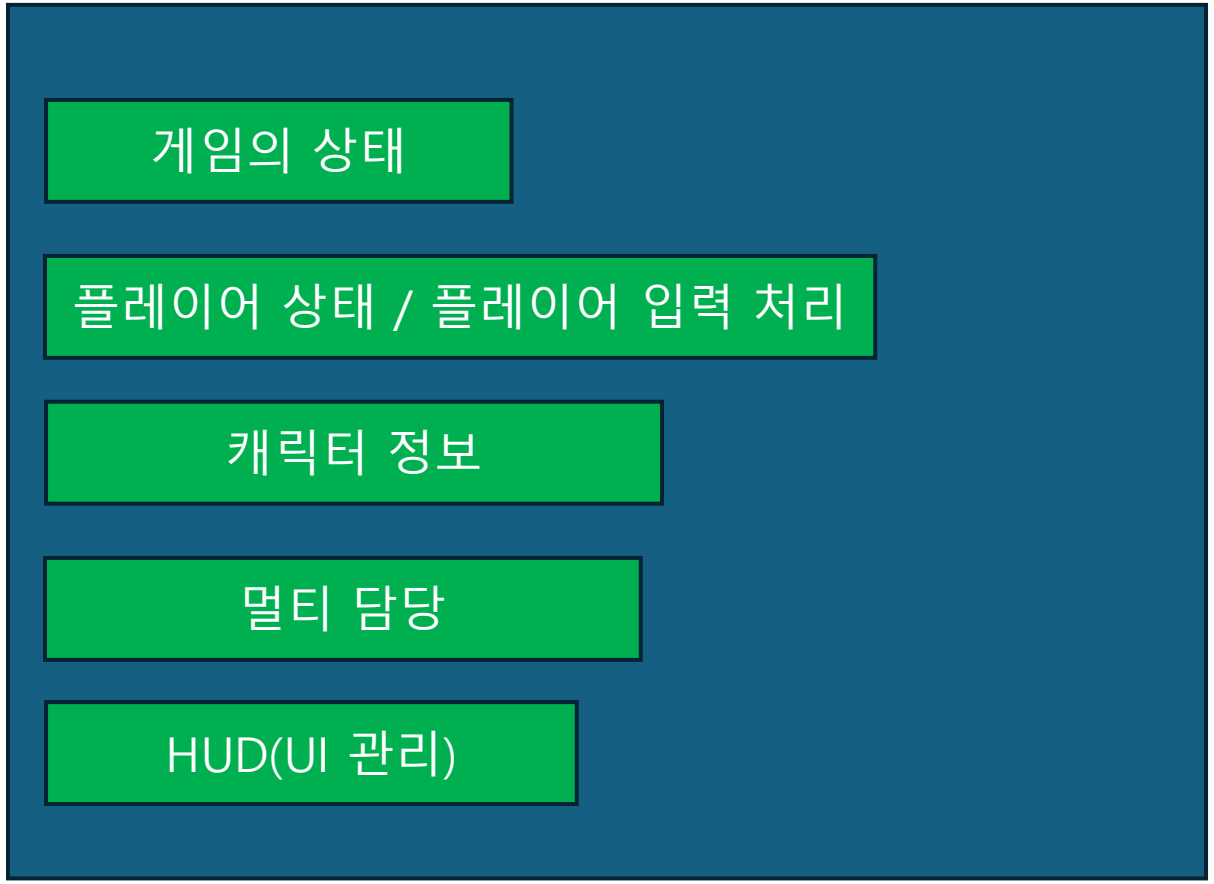
```
AGodManager::AGodManager()
{
    PrimaryActorTick.bCanEverTick = true;

    // — 초기값 설정이 이미 끔찍하게 길다 —
    PlayerGold = 0;
    PlayerGem = 0;
    PlayerHP = 100.f;
    PlayerMaxHP = 100.f;
    PlayerAttack = 10.f;
    PlayerDefense = 5.f;
    PlayerLevel = 1;
    PlayerEXP = 0;
    CurrentChapter = 1;
    MasterVolume = 1.f;
    BGMVolume = 0.8f;
    SFXVolume = 1.f;
    CurrentBGM = nullptr;
    PreviousGold = 0;
    SpeedHackCheckTimer = 0.f;
    bIsGamepadConnected = false;
    CurrentInputMode = TEXT("Keyboard");
    CurrentBoss = nullptr;
    bIsBossFight = false;
    BossPhaseTimer = 0.f;
    PlayerCharacter = nullptr;
    bIsServer = false;
    bIsConnected = false;
    CurrentPlayerCount = 0;
    NetworkLatency = 0.f;
    GraphicsQuality = 2;
    bVSyncEnabled = true;
    TargetFPS = 60;
    bIsRaidActive = false;
    RaidTimer = 0.f;
}

void AGodManager::BeginPlay()
```

- 기능 하나 만들 때마다 여기에 넣어주기?
  - 모든 기능이 다 GameManager을 통하게 만들기?
  - 객체지향 설계를 완전 파괴하는 설계..

그래서, 어떤 매니저를 만들게 될까?



- 나머지 기능들은 이 정도에서 나눠 가져도
- 아무런 문제가 없다!

Problem 3. We Make a Hundred GameManagers!

- 아마 저런 형태로, 개발을 시작할 때마다 매니저를 계속 만들게 될 것이다.

- 규격을 정하지 않고 자유롭게 개발할 여지를 남겨두기
- Vs
- 규격을 정해두고, 향후 나오는 모든 기능들을
- 그걸 기준으로 업데이트하기

Two Ways

- 모든 관리 주체에 대한 자율성을 보장한 유니티.
- 그러나 언리얼의 설계 철학 – “모두가 통일된 프레임워크로!”
- 왜? 게임이 거대해질수록, 검증된 구조를 쓰는 것이 중요하니까!
  - 대규모 게임용 엔진으로 발전해온 언리얼의 특성 상..

어떻게 “게임을 관리하는 주체” 를 일반화시킬 것인가?

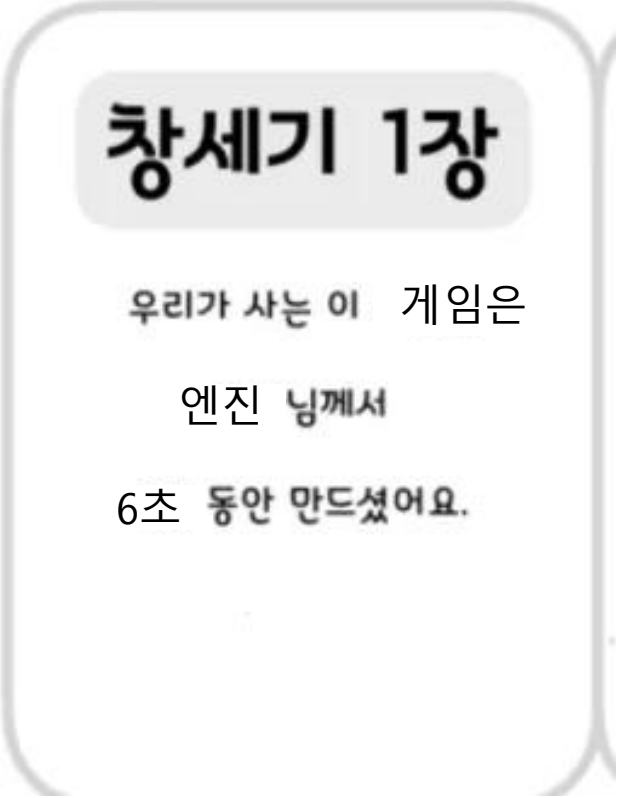
- 조금 재미없을 수도 있는 내용..
- 실습을 조금 늘려보긴 할 것..
- 그래도 지루한 부분을 잘 해놔야
- 나중에 재밌는 걸 할때
- 막힘이 없다

날짜 (월요일)	강의 내용	시간	비고
	언리얼 엔진의 기초, 기본 문법, 그리고 상속에 대해서	18:30 ~ 20:00	엔트리 레벨의 핵심 개념
	월드의 생성 과정, Gamemode / GameState, 그리고 라이프사이클.	18:30 ~ 20:00	게임의 기본 구조 이해
	Actor에 대해서 - AActor와 UObject의 이해, 그리고 Pawn과 Character에 대해서, 그리고 PlayerCharacter와 PlayerState에 대해서 (실습)	18:30 ~ 20:00	플레이어 및 캐릭터 관리
	(중간고사 휴강)	휴강	대학 중간고사 기간
	(중간고사 휴강)	휴강	대학 중간고사 기간
	HUD, 그리고 위젯 블루프린트, 그리고 블루프린트의 기초.	18:30 ~ 20:00	UI 및 비주얼 스크립팅 기본
	언리얼 C++ 심화 및 컨벤션: 1. 가비지 콜렉터와 리플렉션, 언리얼식 C++에 대해서. 2. 언리얼 코딩 컨벤션	18:30 ~ 20:00	심화 C++ 및 개발 표준
	설계 방법론 및 통신: 1. 델리게이트. 2. 컴포넌트 기반 설계 방법론	18:30 ~ 20:00	고급 프로그래밍 패턴
	에셋 관리 방법론 1: TObjectPtr, weakptr, classof, Uclass, 그리고 softptr	18:30 ~ 20:00	메모리 및 객체 관리 심화
	에셋 관리 방법론 2: 1. DataAsset, 그리고	18:30 ~	데이터 중심 설계

2. GameMode / GameState

0. Let's Follow 월드에 애들이 스폰되는 과정.

- Empty World -> 가장 기본적인 스폰부터 배웁시다.



모든 코드는 PIE 코드를 참고했습니다!

1. Start Game

엔진

Create!

GameInstance

- 시작한 시간
- 시작할 맵
- 시작할 수 있는가? 에 대한 정보
- Uworld에게 넘겨줄 정보
- 오디오 드라이버 / NetDriver.
- Others...

2. Init Uworld

GameInstance

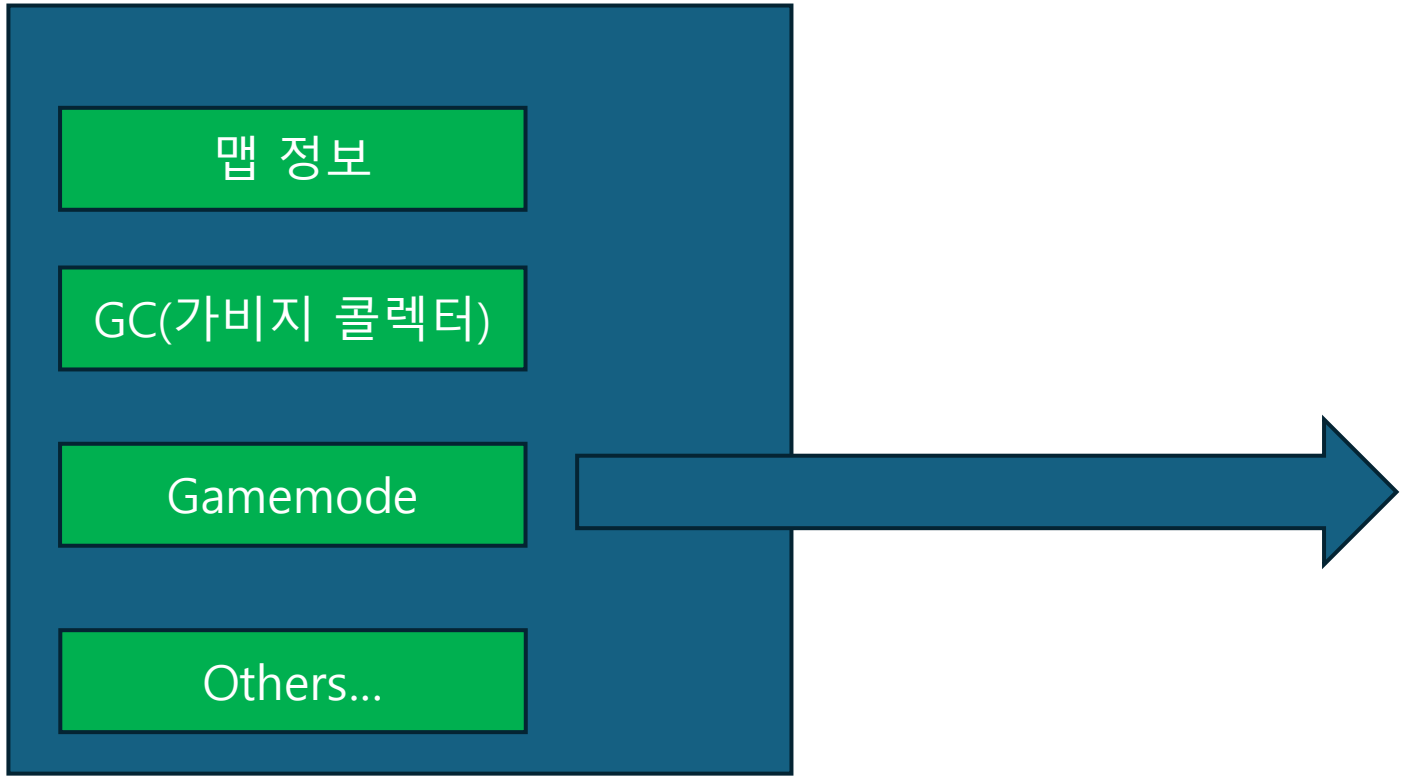
- 시작한 시간
- 시작할 맵
- 시작할 수 있는가? 에 대한 정보
- Uworld에게 넘겨줄 정보
- 오디오 드라이버 / NetDriver.
- Others...

Init!

Uworld

3. Create World's Component!

Uworld



유저가 쓰는 것 중엔
가장 먼저 생긴다!

2. GameMode is Spawned!

Gamemode

- GameState
- PlayerController(플레이어 입력 처리)
- DefaultPawn :
  실제 플레이어가 조종하는 캐릭터
- PlayerState : 플레이어 상태 관리
- HUD(UI 관리)

```
AGameModeBase::AGameModeBase(const FObjectInitializer& Object
    : Super(ObjectInitializer.DoNotCreateDefaultSubobject(TEX
{
    bNetLoadOnClient = false;
    bPauseable = true;
    bStartPlayersAsSpectators = false;

    DefaultPawnClass = ADefaultPawn::StaticClass();
    PlayerControllerClass = APlayerController::StaticClass();
    PlayerStateClass = APlayerState::StaticClass();
    GameStateClass = AGameStateBase::StaticClass();
    HUDClass = AHUD::StaticClass();
    GameSessionClass = AGameSession::StaticClass();
    SpectatorClass = ASpectatorPawn::StaticClass();
    ReplaySpectatorPlayerControllerClass = APlayerController:
    ServerStatReplicatorClass = AServerStatReplicator::Static
}
```

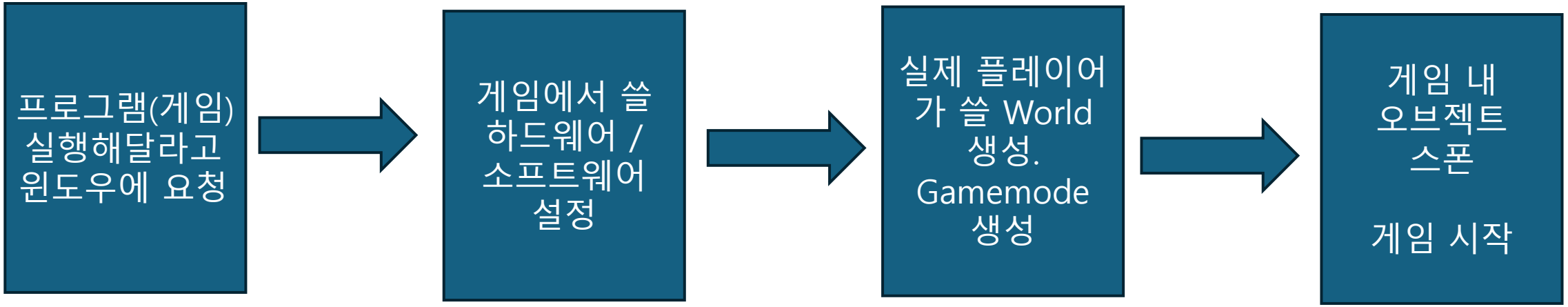
유저에게 관여하는 클래스들을
전부 다 정할 권한이 있음.

3. 이후 과정들.. (순서는 정해져있지 않다)

- GameState 등, 여러 Manager들이 생긴다.
- World에 있는 Object들이 생긴다.
- Object들이 모두 만들어지면, Gamemode에게 시작할 수 있냐고 확인을 받는다.
- 준비가 다 되었으면 시작한다!
- 이후 Gamemode는, 플레이어가 들어오는 것을 지속적으로 확인하고, 누군가 들어왔을 때 처리를 담당한다.

4. 모든 게임은 룰이 필요하다.

- 그리고 우리는 어딘가에서 룰을 주입해줘야 한다.
  - 그리고 누군가는 룰을 들고 있어야 한다.



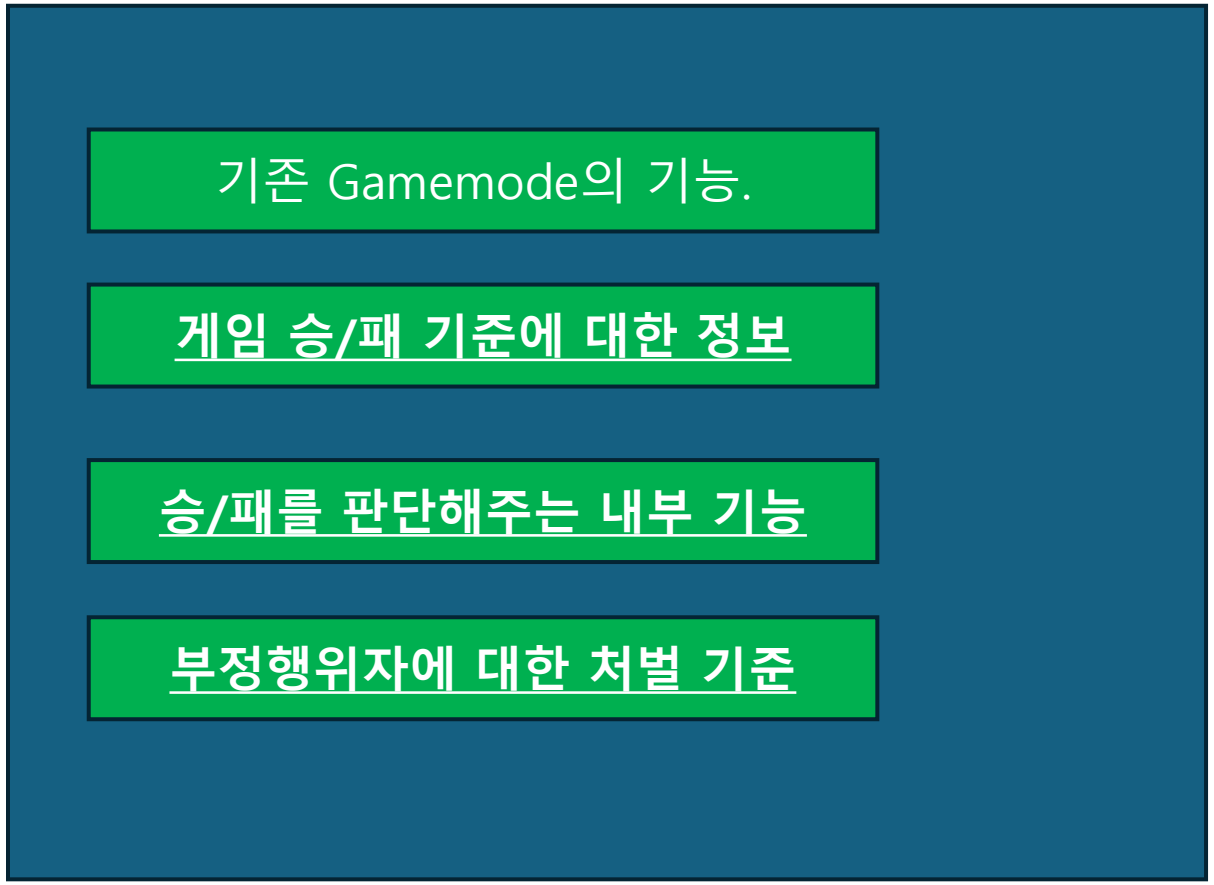
- 누가 룰을 들고 있는 것이 좋을까?
  - 그리고 언제 룰을 넣어주는 것이 나을까?

GameMode = Modern GameManager

- 게임에 필요한 모든 매니저를 지정할 권한.
- 게임 내 오브젝트들 중에서 가장 빠르게 스폰하며
- 모든 매니저들을 알고 있지만
- 본인이 직접 담당하는 일은 플레이어의 접속 관리 정도임
- <u>여기에 추가로 룰 관리 / 감시하는 것 정도를 더 넣어준다면 어떨까?</u>

So We Can Make a Modern GameManager!

Gamemode



- 기존 Gamemode가 하는 일
  - 월드 기본 오브젝트 결정.
  - 플레이어 접속 관리.
    - Very Compact
- 우리가 Gamemode에게 줄 수 있는 일
  - 원칙만 지킨다면, 무궁무진함!
    - 원칙
      - 게임의 룰,
      - 구성에 관련된 정보.
  - This is Modern!

GameMode/GameState(2)

Problem 2. About Multiplayer

- 솔로 플레이라면 이 정도로 되었을지도 모르지만
- 언리얼 엔진은 대규모 멀티를 상정해두고 만든 툴이다.
- 멀티플레이 상황에서도, 과연 이 정도면 될까?

Server / Client

- Server
  - 주인
  - 게임 월드를 연 사람.
  - 관리 권한을 가진 사람.
  - 소유자

- 클라이언트
  - 남의 게임 월드에 참여한 사람
  - 데이터를 받는 입장.
  - 참가자.

서버 – 클라 통신 로직 (간략히)

- 전통적인 방법 : RPC
  - 이 함수 실행해줘
  - 유저가 다른 유저한테 “이거 해줘” 라고 요청을 보내는 것.

- 소실되더라도 빠르게 / 느리더라도 안전하게
  - 게임은 명확히 후자.

- 필요할 때만 계속 데이터를 보내는 것으로
- 게임에 맞는 반응성을 챙길 수 있는가?

The Solution – Replication

- 여기서 중요한건, “진본“ 의 차이 – Authority의 차이.
- 모든 진본은 서버한테 있고, 클라이언트는 그걸 복제받는다.

Problem :
클라이언트는 어디까지 알고 있어야 하지?

- 멀티플레이의 난제 – 어느 정도 데이터까지 복제해야 하지?

- 클라이언트
  - 서버가 준 만큼의 데이터만 가지고
  - 게임을 진행해야 하는 상황

- 어느 정도까지 줘야, 클라이언트가 게임을 하는데 지장이 없지?

Pool 한 해결책 : 서버가 모두 통제해!!

- Only Server Has – Rules and States
- 철통 보안을 지키는? 선택
  - 모든 정보를 서버 쪽이 통제하게 된다

서버

Gamemode(룰)

게임 상태 정보

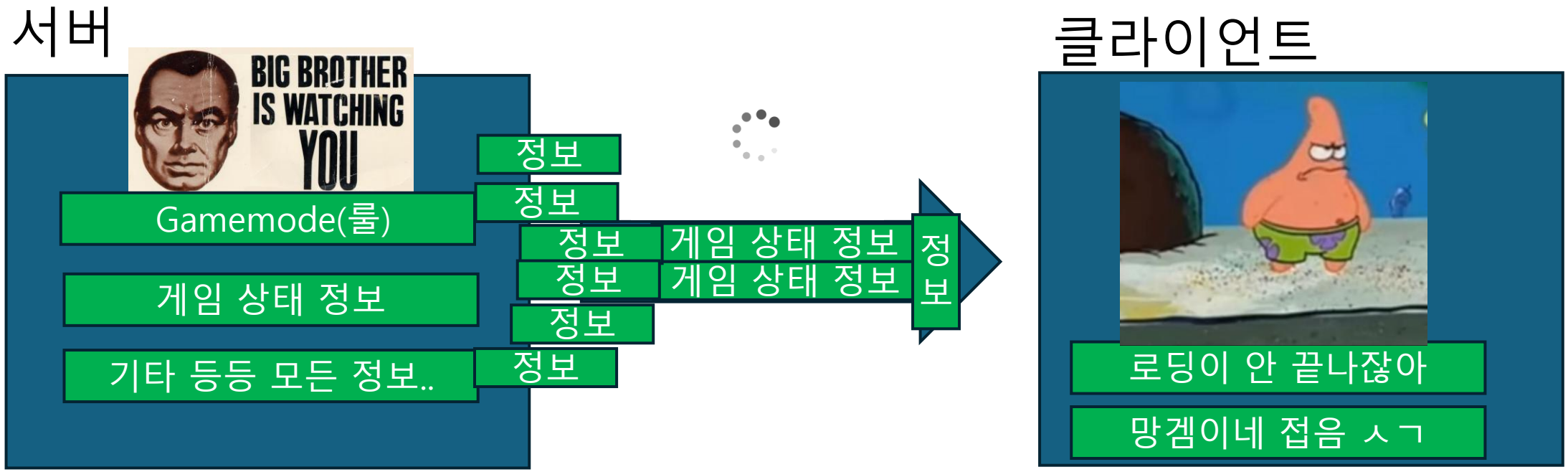
기타 등등 모든 정보..

클라이언트

헿ㅎㅎ게임재밌다

Poor한 해결책: 서버가 전부 통제해!!

- 클라이언트 쪽에서 현재 상태를 알아야 해
  - 어떻게 알 수 있지? 매번 서버 쪽에 요청을 보내?
  - 아니면 매번 서버 쪽에서 데이터를 보내줘?
  - 대역폭은 한정적인데? (병목이 생긴다!)



Poor한 해결책 2. 전부 다 공유하자!!

- 우린 친구니까!

서버 & 클라이언트

Gamemode(룰)

게임 상태 정보

기타 등등 모든 정보..

보안 정보

안티치트 정보

가챠 확률 정보 등..

Poor한 해결책2. 전부 다 공유하자!!

- 공격이 날아온다.
  - 공격자도 데이터를 공유받는 중이니까..

서버 & 클라이언트

Gamemode(룰)

게임 상태 정보

기타 등등 모든 정보..

보안 정보

안티치트 정보

가챠 확률 정보 등..

Pool한 해결책 2. 전부 다 공유하자!!

- Gamemode
  - 시작했을 때, 룰을 알려주고, 룰을 감시하는 역할.
- 만약 룰을 바꿔버린다면?

Smart Solution : 룰 / 상태를 나누자.

- 룰 : 서버만이 알고 있어야 하는 것 (Only Server -> Authority)

- 상태 : 모두가 가지고 있되, 진본은 서버가 가지고 있어야 함
  - 그래서 진위비교가 가능한 것.(Local Authority)
  - 각각의 GameState에 대한 변경 권한은 있지만
  - 변경 요청은 서버를 통해야 함.

```
//=============================================================================
//  GameMode defines the rules and mechanics of the game.  It is only
//  instanced on the server and will never exist on the client.
//=============================================================================
```

GameState : 상태를 가지고 있는다.

- 정보를 '가지고 있는 것' 에 초점을 둔 클래스.

```
UCLASS(config=Game, notplaceable, BlueprintType, Blueprintable, MinimalAPI)
class AGameStateBase : public AInfo
{
    GENERATED_UCLASS_BODY()

public:

    //~==============================================================================
    // General accessors and variables

    /** Class of the server's game mode, assigned by GameModeBase. */
    UPROPERTY(Transient, BlueprintReadOnly, Category=GameState, ReplicatedUsing=OnRep_G
    TSubclassOf<AGameModeBase>  GameModeClass;

    /** Instance of the current game mode, exists only on the server. For non-authority
    UPROPERTY(Transient, BlueprintReadOnly, Category=GameState)
    TObjectPtr<AGameModeBase> AuthorityGameMode;

    /** Class used by spectators, assigned by GameModeBase. */
    UPROPERTY(Transient, BlueprintReadOnly, Category=GameState, ReplicatedUsing=OnRep_S
    TSubclassOf<ASpectatorPawn> SpectatorClass;

    /** Array of all PlayerStates, maintained on both server and clients (PlayerStates
    UPROPERTY(Transient, BlueprintReadOnly, Category=GameState)
    TArray<TObjectPtr<APlayerState>> PlayerArray;
```

```
    /** Returns the simulated TimeSeconds on the server, will be synchronized on client
    UFUNCTION(BlueprintCallable, Category=GameState)
    ENGINE_API virtual double GetServerWorldTimeSeconds() const;

    /** Returns true if the world has started play (called BeginPlay on actors) */
    UFUNCTION(BlueprintCallable, Category=GameState)
    ENGINE_API virtual bool HasBegunPlay() const;

    /** Returns true if the world has started match (called MatchStarted callbacks) */
    UFUNCTION(BlueprintCallable, Category=GameState)
    ENGINE_API virtual bool HasMatchStarted() const;

    /** Returns true if the match can be considered ended. Defaults to false. */
    UFUNCTION(BlueprintCallable, Category = Game)
    ENGINE_API virtual bool HasMatchEnded() const;

    /** Returns the time that should be used as when a player started */
    UFUNCTION(BlueprintCallable, Category=GameState)
    ENGINE_API virtual float GetPlayerStartTime(AController* Controller) const;

    /** Returns how much time needs to be spent before a player can respawn */
    UFUNCTION(BlueprintCallable, Category=GameState)
    ENGINE_API virtual float GetPlayerRespawnDelay(AController* Controller) const;
```

복제된다 – GameState의 핵심 특징..

- 서버/클라이언트가 거의 항상 같은 정보를 공유받고 있는다.
  - 정보 업데이트의 우선순위가 항상 최상위임

```
AGameStateBase::AGameStateBase(const FObjectInitializer& ObjectInitializer)
  : Super(ObjectInitializer
      .DoNotCreateDefaultSubobject(TEXT("Sprite")) )

  SetRemoteRoleForBackwardsCompat(ROLE_SimulatedProxy);
  bReplicates = true;
  bAlwaysRelevant = true;
  SetReplicatingMovement(false);

  // Note: this is very important to set to false. Though al
  // they are held on to and brought over into the new world
  // and given static NetGUIDs. This also causes their delet
  // may end up deleting the new PlayerStates they had just
  bNetLoadOnClient = false;

  // Default to every few seconds.
  ServerWorldTimeSecondsUpdateFrequency = 0.1f;

  SumServerWorldTimeSecondsDelta = 0.0;
  NumServerWorldTimeSecondsDeltas = 0;

  NetPriority = 10.f;  / We need to prioritize updates to en
```

```
void AActor::InitializeDefaults()
{
    PrimaryActorTick.TickGroup = TG_PrePhysics;
    // Default to no tick function, but if we set 'never ticks' to false (so there is
    PrimaryActorTick.bCanEverTick = false;
    PrimaryActorTick.bStartWithTickEnabled = true;
    PrimaryActorTick.SetTickFunctionEnable(false);
    bAsyncPhysicsTickEnabled = false;

    CustomTimeDilation = 1.0f;

    SetRole(ROLE_Authority);
    RemoteRole = ROLE_None;
    bReplicates = false;
    bCallPreReplication = true;
    bCallPreReplicationForReplay = true;
    bReplicateUsingRegisteredSubObjectList = GDefaultUseSubObjectReplicationList;
    PhysicsReplicationMode = EPhysicsReplicationMode::Default;
    NetPriority = 1.0f;
```

GameState를 통해 얻을 수 있는 정보

- 게임이 시작했는가?
  - 왜냐면 : 게임 시작 준비가 되었을 때 활성화되는게 GameState라서.
- 이 플레이어는 서버인가, 클라이언트인가?
- 다른 Manager 클래스들의 참조
- 서버 시간은 어떻게 되는가? / 현재 접속한 플레이어는 누구누구인가? / 등등..

핵심 – 그리고 그 정보는 서버랑 공유된다

- 서로 필요한 정보만 가지고 있고
  - 민감한 정보는 서버가(Gamemode), 공유해야 하는 정보(GameState)는 모두 복제해서 가지고 있는다!

서버

- Gamemode(룰)
- GameState(상태)
- 보안 관련 데이터
- 확률 / 개인정보 데이터

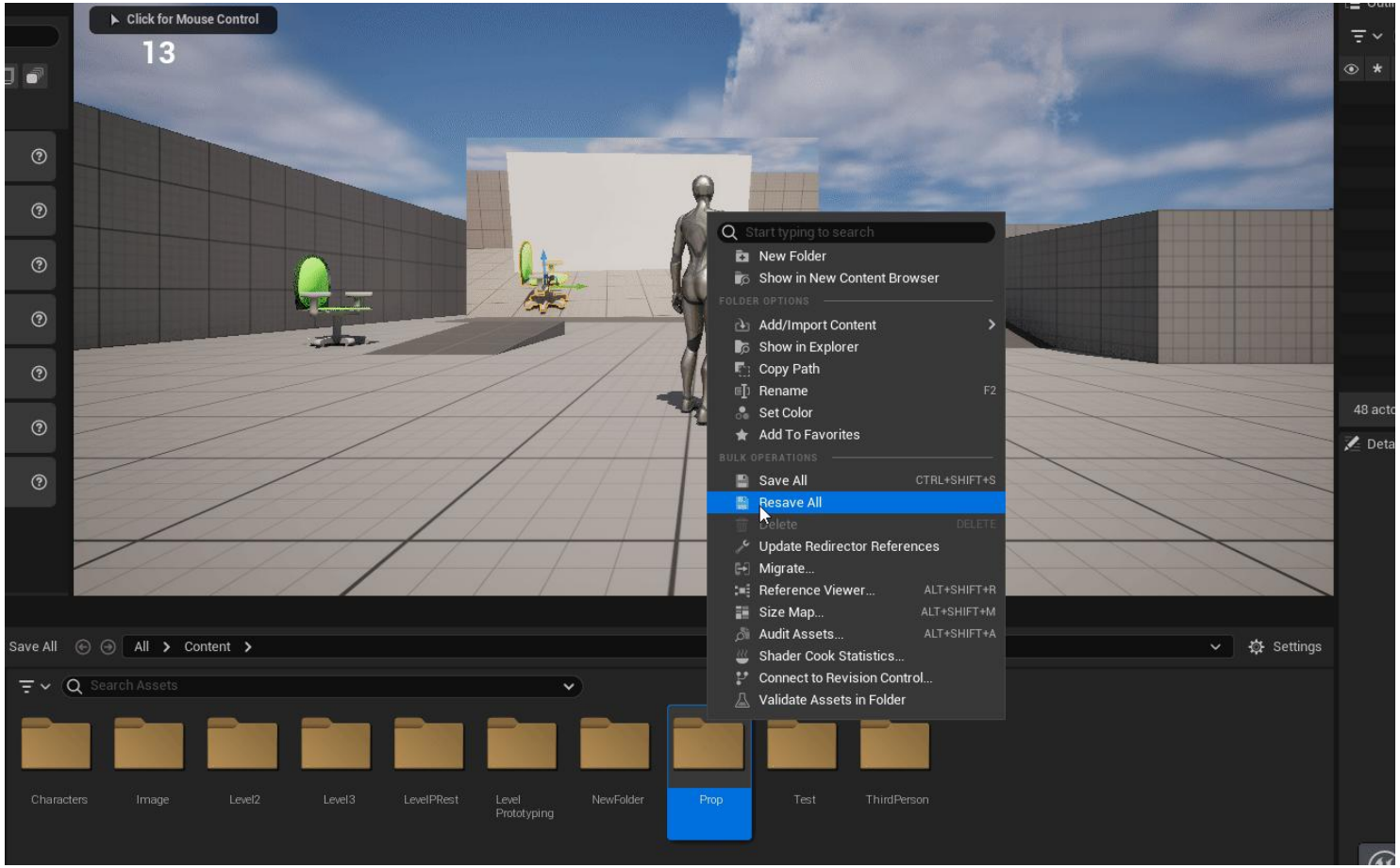
클라이언트

- GameState(복제)
- 캐릭터 상태
- 맵 / 오브젝트 등 데이터
- 게임 진행 상태 등..

What are **Actors** and **Players**?

- 이론적인 부분에서 중요한 것들을 다뤄보았다.
  - 잘 알아두면 나중에 헷갈리지 않지만
  - 재미없는 내용이긴 하다..
  - 실습을 나가기에도 뭐하고

- 실제 게임에서 중요한 것 –
  - Actor가 움직이는 것, 플레이어가 움직이는 것!

Next Class?

- 실제로 만들어보자! / 룰이 있고, 플레이어가 움직이는 게임